Muskellunge Lake Resort

The resort app — a product & engineering walkthrough

What it is · every feature · how it's built · where it could go next

EST. 1987 · Tomahawk, Wisconsin · A Next.js 16 / React 19 PWA

Prepared for a fellow programmer — skim the product half, then dig into the backend.

Contents

How to read this

Sections 1–7 are the product — what the app does and how it feels to use. Sections 8–11 are the engineering — stack, database, security, and deploy. Section 12 is a short list of trade-offs I'd genuinely like a second opinion on.

SECTION 1

What it is & who it's for

A private, mobile-first app for one family's Northwoods resort — and the week-long reunion at the heart of it.

Muskellunge Lake Resort (MLR) is a real family resort near Tomahawk, Wisconsin (the original light-housekeeping cabins on the lake, EST 1987). This app is a small, private **progressive web app** for the family and community around it — think "a tiny private social network + event tool," not a public booking site. Anyone with the link can browse; a **verified email unlocks interaction** (posting, RSVP, reactions).

It does double duty. Most of the year it's the **year-round resort app** — activities, dining, amenities, committees, tee-time booking. Once a year it becomes the home base for **Family Fest**, a week-long family reunion, which lives *inside* the same app as a themed section at `/family-fest`.

The two halves, one app

	Year-round MLR	Family Fest section
Look	Forest-green, Northwoods heritage	Renaissance / parchment (scoped theme)
Lives at	`/, /posts, /profile, …`	`/family-fest/*`
Content	Activities, dining, committees, tee times	Schedule, dinners, dues, the week itself
Display font	Yellowtail script wordmark	Cinzel Roman-serif titles

The one trick that ties it together

Rather than build two apps, the Family Fest is modeled as a **"season" of the resort** that rises and recedes through the year — **off-season » planning » live » wrap**. The home screen, the tab bar, and the fest hub all reshape themselves around that phase. Full detail in Section 7.

Current status

The backend is **live**: passwordless auth, member profiles, and the shared photo/video feed all run on Supabase, with media files self-hosted on a Mac mini. A `READ_ONLY` feature flag still gates a few not-yet-backed features (in-app committee join, RSVP writes) behind a tasteful "coming soon," so nothing ever looks broken. It's deployed on both **Vercel** and **GitHub Pages**.

SECTION 2

The map — navigation & every screen

Four bottom tabs carry the whole app; a few Home cards reach the year-round extras. Nothing is ever more than one tap from home.

The four tabs (bottom nav)

Tab	Route	What's there
Home	`/`	Logo, share-the-app, the Family Fest season card, dues CTA, resort cards, tee-time link
Posts	`/posts`	The shared family feed — photos, videos, comments, reactions, tagging, a photo timeline
Family Fest	`/family-fest`	The whole week in one view; wears a live "pulse" dot during the event
Profile	`/profile`	Your identity, avatar, contact/pay info, email-alert toggle, admin tools, sign out

The `TABS` array in `TabBar.tsx` is the single source of truth for the nav. The Family Fest tab is colored heraldic-wine (not green) so it reads as its own world, and shows a pulsing red dot during the live and wrap phases.

Reached from the Home screen (not tabs)

Destination	Route	Notes
Activities	`/activities`	Resort things-to-do in 4 color-coded categories (water / land / kids / evening)
Dining & amenities	`/dining`	Eat-and-drink venues + practical amenities (WiFi, check-in, laundry…)
Committees	`/committees`	Volunteer groups; tap one » roster with call/text/email + "request to join"
Work Weekends	`/work-weekends`	Season-prep weekends (hidden during the fest week)
Tee Times	`/tee-times`	Deep-links into Inshalla CC's foreUP booking, pre-filtered to a day

Inside Family Fest — the click-throughs

The fest hub is one flowing page (no sub-nav), built to answer "what's happening, and what do I do today?":

- **FestStatus** at the top — during the week it shows **Day n of N** and every one of today's events in full (time, place, what to bring, who's in charge with tap-to-call/text), plus tonight's dinner and head chef.
- **FestWeek** below — an "anytime all week" list (e.g. the scavenger hunt) plus a day-by-day accordion. Tap a day to expand its events and dinner; each row links to a detail page.

- **Detail pages** — `/family-fest/schedule/[id]` (an event) and `/family-fest/dinners/[id]` (a dinner: menu, prep vs. serve time/place, the cooking “houses,” chef contact).
- **/family-fest/pay** — the dues amount + organizers to pay by Venmo/Zelle.

> **Navigation principle**
>
> Detail pages carry an iOS-style **‹ back** link that always names its destination, and the bottom tab bar is always present — so you can never get stranded.

SECTION 3

Sign-in & identity

Public to browse, verified to act. No passwords — a 6-digit email code and a session that sticks.

Identity is **on-demand, not a gate**. The entire app is browsable signed-out. The moment you try to *do* something — post, react, RSVP — a sign-in sheet slides up. That's `promptSignIn()` from a small React context (`IdentityProvider`) that any component can call.

The flow

- **Enter name + email** » Supabase `signInWithOtp()` emails a 6-digit code. The name is passed as user metadata so a DB trigger can seed the profile.
- **Enter the code** » `verifyOtp()` creates a real session. Supabase persists the refresh token on-device and auto-refreshes it, so you **stay logged in** — a new code is only needed on a new device.
- **On every load**, the provider restores the session and loads the member's row from the `profiles` table (display name, avatar, alert opt-in, admin flag).

Admins

Admin is a `profiles.is_admin` boolean in the database (set from the SQL editor, never from the client), with a tiny bootstrap allow-list in code so there's an admin from day one. Admins get an in-app **alert composer** on their Profile (Section 6).

> **Why this shape**
> It's deliberately not high-security — the only requirement is "a real, verified email to interact." Email OTP confirms the address, the saved session keeps it effortless for the older relatives, and there are no passwords for anyone to forget. Build-safe, too: if Supabase env vars are absent, sign-in simply isn't offered and the app stays fully browsable.

SECTION 4

The Posts feed in depth

The social heart of the app — a shared, cross-device feed that behaves like a small private Instagram for the family.

`PostsView.tsx` is the largest component in the app. It backs the Posts tab and powers:

- **A collapsing composer** — starts as a single “Share something…” box; on focus it reveals photo/video attach, people-tagging, and a backdate control.
- **Multi-photo & video posts** — a swipeable, snap-scrolling carousel with dot indicators and an n/N counter; tap a photo for a full-screen lightbox.
- **Tagging** — search the family roster and tag people; a “ Tagged me” filter shows posts you’re in.
- **Comments & emoji reactions** — six reactions, one per member per post (upsert); tap a reaction chip to see who reacted.
- **A photo timeline** — posts group by day with “Today / Yesterday / date” headers, plus month chips and a date picker to jump back. Posting late? **Backdate** a photo to *when it happened* (a separate `occurred_at`) so it slots into history.
- **Inline editing** — author or admin can edit text, add/remove media, retag, move the date, or delete — saved as a diff.
- **Share out** — the native Web Share sheet (» Instagram, Messages…), falling back to the family Facebook group.

How it stays live

The feed subscribes to Supabase **Realtime** on five tables (`posts, post_media, post_comments, post_reactions, post_tags`); any change re-queries the feed, so a photo posted on one phone appears on everyone’s. Display names and avatars are resolved from a separate `profiles` query and joined client-side — so renaming yourself updates your name on *every* past post and comment retroactively.

Media handling

Photos are **compressed in-browser** to ~1920px JPEGs via canvas before upload (smaller, faster, and it sidesteps iPhone HDR/HEIC display quirks); videos upload as-is. Uploads stream to the Mac-mini media server over `XMLHttpRequest` (chosen over fetch for a real progress bar). If there’s no backend, the feed degrades to seed posts + local-only posting so the UI is never empty.

SECTION 5

Contact & pay — the member card

Tap anyone's name or avatar, anywhere, to reach or pay them — with their own preferred method floated to the top.

Every name and avatar in the app — a post author, a comment, a tagged person — is tappable. It opens `MemberSheet`, a drag-to-dismiss bottom sheet that reads that member's optional contact and payment fields from their profile and turns them into one-tap actions:

- **Contact** — Text (`sms:`), Call (`tel:`), Email (`mailto:`).
- **Pay** — Venmo (deep-links straight to the pay screen), Zelle (copy handle), Apple Cash (via Messages), Cash App, PayPal — each as a brand-colored button.
- **Preferred method** — each member marks how they'd *like* to be reached/paid; that option is moved to the front and flagged.

Members fill these in (all optional) under **Profile » Contact & payment**. Phones are stored E.164 so `tel:`/`sms:` work on both iPhone and Android. No payment credentials ever live in the app — buttons just hand off to the user's own Venmo/bank/Messages.

> **Nice touch**
> The same contact pattern powers committee rosters and the Family Fest "who's in charge" / "head chef" lines — it's one helper (`lib/contact.ts`) reused everywhere.

SECTION 6

Announcements & alerts

A dismissible notice banner at the top of every screen — with an admin composer to push to it.

`AnnouncementBanner` renders at the top of the app on every page. It merges two sources: server-fed announcements (today seed data; the seam is built for a Google-Drive feed later) and **admin-posted local alerts**. Notices come in two severities — a loud alert and a quiet ■ info — and each is **dismissible per-device** (remembered in `localStorage`) so it never nags after it’s read.

Admins get a composer on their Profile (title, body, severity » Push). Today it writes to local storage and fires an event so the banner updates instantly across tabs. The **backend seam** is already designed: the same action will validate the admin server-side, broadcast to all devices, and email the members who opted into alerts.

SECTION 7

The Family Fest “season” engine

One small pure function decides how the whole app behaves on any given day of the year.

This is the conceptual core. `getFestSeason(start, end, now)` in `lib/festSeason.ts` is a pure, dependency-free function that maps today’s date to a **phase** and a bag of derived facts (day number, days-until, wrap days left…):

Phase	When	What the app does
off-season	> 60 days out	A quiet Family Fest banner on Home; everything resort-normal
planning	60 days » start	Partial takeover: rally volunteers, preview the plan, push dues
live	during the week	Full takeover: “Day n of N + today’s events,” a live dot on the tab, resort cards recede
wrap	14 days after	Takeover lingers, nudging “post the photos you didn’t get to”

Why it’s computed on the client

The phase depends on “now,” and the app ships as a **static export** to GitHub Pages. A build-time `new Date()` would freeze the phase at deploy time. So a `useFestSeason` hook computes it in the browser and returns `null` until mounted — which also avoids a hydration mismatch. The same code is correct on the static Pages build and on Vercel.

Demo time-travel (a developer favorite)

A `DemoDateProvider` lets you set “see the app as if it’s this day,” persisted in localStorage and exposed on the Profile screen with quick-jumps (Run-up / Day 1–7 / After). Every season-aware surface respects it, so you can preview the live-week takeover in the middle of winter. The countdown component reads the same override.

Note: the constants live in two repos. The fest started as its own standalone app, so `festSeason.ts` and the event seed data are *mirrored byte-for-byte* in a `family-fest` repo; the standalone is being retired into this section.

SECTION 8

Technical architecture

Next.js 16 App Router, React 19, Tailwind v4, TypeScript — a mobile-first PWA with a CSS-token theme system.

Stack

Layer	Choice	Notes
Framework	Next.js 16 (App Router)	Server components by default; client components opt in with “use client”
UI	React 19 + TypeScript	Strict types; shared domain shapes in lib/types.ts
Styling	Tailwind v4	Theme tokens as CSS variables in an @theme block — no hard-coded hex
Backend	Supabase	Postgres + Auth (email OTP) + Realtime; @supabase/supabase-js client
Media	Self-hosted Express	On a Mac mini behind a Tailscale Funnel (Section 10)
Hosting	Vercel + GitHub Pages	Same code, two build targets (Section 11)

Rendering model

Pages are server components that pass static seed data into client components; anything time- or auth-dependent is computed client-side behind a *mounted gate* (return null until mounted) to keep the static export and the first client paint identical. Dynamic routes (`/committees/[slug]`, the fest detail pages) use `generateStaticParams()` so they pre-render for the static export.

Theming & the scoped sub-theme

All color is CSS variables in `app/globals.css`; Tailwind v4 turns each `--color-*` token into `bg-*/text-*/ring-*` utilities. The clever bit: the Family Fest section’s parchment palette and Cinzel serif are scoped by **re-declaring those same CSS variables** under a `.ff-section` wrapper. Every Tailwind utility inside that subtree instantly renders in heraldic wine/parchment, while the forest-green resort chrome above and below is untouched — a whole theme swap with zero component changes.

```
/* globals.css */
@theme {
  --color-primary: #15503a;   /* forest green */
  --color-background: #f6f6f1;
}
.ff-section {                 /* the Family Fest subtree */
  --color-primary: #8b2e2e;   /* heraldic wine */
  --color-background: #f4ecd8;/* parchment */
  --font-display: var(--font-cinzel), serif;
}
```

Conventions worth noting

- **Light mode only, forever** — there’s an explicit guard against dark translucent surface tints (they go muddy grey on the light bg); a recurring lesson written into the CSS.
- **One formatter module** — every date/time/currency string goes through `lib/format.ts` so the whole app reads consistently.
- **A small GPU-only motion system** — transform/opacity keyframes (page-enter, sheet, pop, rise) with a `prefers-reduced-motion` collapse to a 1ms fade.
- **PWA** — web manifest, installable, iOS “Add to Home Screen” hint, safe-area insets for notches.

SECTION 9

Data & backend — Supabase

One Postgres database, one identity, row-level security on every table, and realtime on the social tables.

Static resort content (activities, dining, schedule, dinners, committees, dues) lives as typed seed data in `lib/data.ts` — swappable for an API later without touching pages. Everything multi-user is in Supabase. The schema is seven idempotent SQL migrations:

Table	Holds	Realtime
profiles	One row per auth user: display_name, avatar_url, is_admin, email_alerts, + contact/pay fields	—
posts	Feed posts: author, text, created_at, occurred_at (timeline)	yes
post_media	Multiple photos/videos per post (storage_path, type, position)	yes
post_comments	Comments, by author	yes
post_reactions	One emoji per member per post (PK = post+user)	yes
post_tags	Tagged members per post	yes
albums	Optional post grouping	yes

Row-level security (the real access model)

Every table has RLS enabled with a consistent posture: **public read** (anyone with the link can browse), **insert your own rows only** (`auth.uid() = author_id`), and **delete/edit your own or admin**. Privilege escalation is blocked at the database: clients are `GRANT`ed update on specific profile columns only — never `is_admin` — so admin can't be self-assigned from the client.

```
-- representative policy (posts)
create policy "posts: insert own" on public.posts
  for insert with check (auth.uid() = author_id);
create policy "posts: delete own or admin" on public.posts
  for delete using (
    auth.uid() = author_id
    or exists (select 1 from profiles p
               where p.id = auth.uid() and p.is_admin));
```

Triggers & identity glue

- A `handle_new_user()` trigger auto-creates a profile on sign-up, seeding `display_name` from the entered name (or the email local-part) and `contact_email` from the auth email.
- An `updated_at` trigger keeps timestamps fresh.
- Migrations are written idempotently (`if not exists`, `drop policy if exists`) and the app degrades gracefully if a migration hasn't run yet (e.g. the timeline's `occurred_at` falls back to `created_at`).

SECTION 10

Self-hosted media server

Login and data stay in cloud Supabase; the photo and video files live on a Mac mini at the house.

To dodge cloud-storage size caps (and cost) on a media-heavy family feed, post photos/videos are stored on a small **Express server running on a Mac mini**, fronted by a stable HTTPS tunnel (Tailscale Funnel). The app stores the returned file URL in `post_media.storage_path`.

How it’s secured

- **Uploads are gated to signed-in family.** The server takes the caller’s Supabase access token and validates it against the cloud project (`/auth/v1/user`) before accepting a file — no secrets needed on the mini.
- **Reads are public** (anyone with the app can view photos), served via `express.static` with HTTP Range support, so videos seek/stream and files cache for a year.
- **Endpoints:** `POST /upload` (auth), `GET /f/<name>` (public), `GET /health`. Filenames are random UUIDs; per-file size cap is configurable.

It also serves the pay-method logos at `/assets`. Ops are handled with a launchd agent + a little “MLR Media Server” control-panel app, with request logging (`[req]/[upload]/[auth]`) for diagnosing upload issues. The **one constraint**: the public URL must stay constant, because it’s baked into stored links.

SECTION 11

Hosting, infrastructure & resources

One codebase, two build targets, and a deliberately frugal mix of free tiers plus a single home server. Here's every moving piece and what it costs.

Dual-target build

`next.config.ts` branches on a `PAGES_BASE_PATH` env var, so the *same* codebase produces two builds:

- **Default (Vercel):** served at the root with cache-control headers (always-revalidate, so a fresh deploy is picked up immediately on an installed PWA). This is the primary, full-featured target.
- **GitHub Pages:** `PAGES_BASE_PATH=/mlr-app` switches Next.js to `output: "export"` — a fully static site under that subpath, with unoptimized images and trailing slashes. A free, always-on mirror of the browse experience.

Pages auto-deploys on push to `main` via GitHub Actions; Vercel is a manual `vercel --prod`. The two live at `mlr-app-omega.vercel.app` and `btheis15.github.io/mlr-app`.

Where each piece runs

Concern	Runs on	Notes
Web app / UI	Vercel (primary) + GitHub Pages (mirror)	Static + client-rendered; CDN-delivered
Database	Supabase Postgres (cloud)	profiles, posts, media metadata, comments, reactions, tags
Auth	Supabase Auth (cloud)	Email OTP; delivery via Gmail/Resend SMTP
Live updates	Supabase Realtime (cloud)	WebSocket postgres-changes on the social tables
Photo/video files	Mac mini (home) + Tailscale Funnel	Express server; disk is the storage limit
Pay/brand assets	Mac mini (/assets)	SVG logos served off the mini, not the cloud
Fonts	Self-hosted via next/font	Yellowtail + Cinzel bundled at build (works offline/PWA)
Keep-alive cron	GitHub Actions	Read-only ping every ~3 days so Supabase doesn't pause

Resources & what they cost

The whole thing is built to run at essentially **$0/month** on free tiers, with one home machine doing the heavy media lifting:

Resource	Tier / spec	Cost
Vercel	Hobby plan — static + edge, generous bandwidth	**free**
GitHub Pages + Actions	Static hosting + CI minutes (deploy + cron)	**free**

Resource	Tier / spec	Cost
Supabase	Free project: ~500 MB Postgres, Auth, Realtime; pauses after 7 days idle	**free**
Email (OTP)	Gmail SMTP (or Resend free, 3k/mo) — one code per login	**free**
Tailscale Funnel	Stable public HTTPS tunnel to the mini	**free**
Mac mini	Always-on box at the house: Express + disk for all media	electricity
Domain	Uses *.vercel.app / *.github.io today; custom domain optional	**optional**

Mac mini operations

The media server is the one piece that isn't a managed cloud service, so it's hardened for unattended uptime: it runs as a **launchd agent** (auto-starts on login, restarts on crash), `caffeinate` keeps the mini awake, and there's a small "MLR Media Server" control-panel app for start/stop + a live request log. The dev/build clone lives at `~/mlr-build` (kept off the iCloud-synced Desktop, which corrupts git repos).

Keeping a seasonal app awake

Supabase's free tier pauses a project after a week of inactivity — and this app is quiet for most of the year. A **GitHub Actions cron** does a read-only ping every ~3 days to keep it alive. It runs on GitHub's cloud (not the mini), never writes anything, and is inert until the secrets are set. Note the split-brain by design: even if the database pauses or the mini sleeps, the static browse experience on Vercel/Pages keeps working — only interaction (posting, sign-in) depends on the cloud + mini.

SECTION 12

Ideas to kick around

Honest trade-offs and rough edges — the parts where I'd most value your read. None are on fire; all are real.

Performance & data

Realtime feed does a full re-query on every change

PERF

Each insert/edit/reaction on any of five tables triggers a complete `refetch()` of the whole feed (six queries). Fine for a family-sized feed; it won't scale to thousands of posts and can thrash during a busy event evening. Options: apply granular realtime payloads to local state, paginate/window the feed, or debounce refetches. Worth it, or premature for this audience?

Sequential uploads & N+1 inserts

PERF

Media files upload one-by-one and each `post_media` / `post_tags` row is inserted in its own round-trip. Batching the inserts and uploading in parallel (with a concurrency limit) would speed up multi-photo posts noticeably on phone networks.

No image CDN / responsive sizes

PERF

Photos are client-compressed to ~1920px and served at full size from the mini for every view, including tiny grid thumbnails. A thumbnail tier (or a CDN/transform in front of the mini) would cut data use a lot on the feed. Is a CDN worth adding given everything else is free-tier?

Security & privacy

"Public read" exposes member contact info

SECURITY

Every `profiles` column is world-readable by design (the app is link-private, not auth-gated). But that now includes phone, email, Venmo/Zelle/PayPal handles. The product intent is per-field visibility (public / members-only / private) — it's specced but not built. Given it's a small trusted family, is column-level RLS (members-only for contact fields) the right next step, or overkill?

Media server trusts any valid token

SECURITY

Upload auth checks only that the bearer token is a valid Supabase user — not that they're a member in good standing, and there's no per-user rate limit or content-type allowlist beyond the size cap. Low risk for a private link, but a cheap hardening target.

Architecture & ops

The Mac mini is a single point of failure

OPS

Photos live on one machine behind a home tunnel; if it sleeps, loses power, or the tunnel URL ever changes, stored links break (and there's no off-box backup beyond Time Machine). It's a great cost play — but would you keep it, or move to cheap object storage (R2/B2) with the mini as a cache?

Two flags to reason about: READ_ONLY vs isSupabaseConfigured

CLEANUP

Feature availability is gated by both a hand-set `READ_ONLY` boolean and a runtime `isSupabaseConfigured` check. Now that auth is live, the two can disagree in confusing ways. Collapsing to one source of truth (derive read-only from config + per-feature capability) would simplify the mental model.

Mirrored season code across two repos

CLEANUP

`festSeason.ts` + event seed data are duplicated byte-for-byte in a separate `family-fest` repo. The merge into this app is done; retiring that repo (or extracting a shared package) removes a real drift risk.

The big-picture question

The roadmap still wants a member directory + "email everyone by name," a committee request»approve loop with live rosters, Google-Drive-fed announcements, and Android web-push. All are designed as isolated backend seams. **If you were picking the next thing to build — for maximum payoff to a non-technical family — what would it be?**

Thanks for reading. Happy to walk through any part of the code — it's all TypeScript, and the README / CLAUDE.md / NEXT-STEPS.md in the repo go deeper on each seam.